DOCUMENTATION · RUN IT

Deployment

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=DEPLOYMENT. This copy was made on 4 November 2026.

A production-shaped setup: the published image, a real reverse proxy terminating TLS, secrets kept out of the compose file, a persistent volume with a backup job, and Prometheus/Grafana wired to the metrics QueryAPIGate already exposes. `docker-compose.yml` in the repo root is a **one-command demo** (open to localhost, a throwaway Postgres, a demo API key) - not this. Start from the compose file below instead.

If you're installing from source or pip rather than Docker, see Installation and setup instead; the gunicorn command and every environment variable there apply here too - this page is about the container and everything around it.

1. The image

```shell
docker pull ghcr.io/anantharajuc/queryapigate:latest
```

TAG	CONTENTS
`X.Y.Z`, `latest`	QueryAPIGate with every optional feature: the PostgreSQL, MySQL, ClickHouse, DuckDB and MongoDB drivers (SQLite is built in), `allowed_tables`, password encryption, signed-in users (JWT), the Redis cache and `queryapigate mcp`
`X.Y.Z-h2`, `latest-h2`	The same plus Java and the H2 driver - also the variant for a generic `jdbc` connection (mount your vendor's jar)

Pin to `X.Y.Z` in production, not `latest`. An upgrade should be a deliberate, one-line version bump you control, not something that happens on the next `docker compose pull`. Bump it the same way you'd bump any other dependency - read the Changelog first, especially around a minor version bump (this project's pre-1.0 equivalent of a major one).

The image already does the things a production container should: runs as a non-root user (`uid 1000`), ships a `HEALTHCHECK` against `/health`, and starts gunicorn with `--worker-class gthread` and one worker (see Scaling out below) - none of that needs reconfiguring.

The store lives in `/data`. A named volume (`-v queryapigate-data:/data`) works as it is; a folder on the host (`-v "$PWD/data:/data"`) has to be created first (`mkdir data`) and be writable by uid 1000 - one Docker creates for you is root's, which the container can't write to (or run with `--user "$(id -u):$(id -g)"`). A starter store: `docker run --rm -v queryapigate-data:/data ghcr.io/anantharajuc/queryapigate queryapigate init`. To build the image yourself: `docker build -t queryapigate .`, with `--build-arg WITH_H2=true` for the H2 variant.

2. Compose file

yaml

```yaml
# docker-compose.yml
services:
  queryapigate:
    image: ghcr.io/anantharajuc/queryapigate:1.2.3   # pin a real version - see above
    restart: unless-stopped
    env_file: .env                                    # secrets live here, never inline - see below
    environment:
      QUERYAPIGATE_TRUST_PROXY: "1"                   # one hop: the reverse proxy below
    volumes:
      - queryapigate-data:/data
    networks:
      - internal
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M

  # Reverse proxy: TLS termination, the only container with a published port. Caddy shown here for its
  # automatic HTTPS; nginx/Traefik work the same way - proxy to queryapigate:5000 with the real client
  # address forwarded.
  proxy:
    image: caddy:2-alpine
    restart: unless-stopped
    ports:
      - "443:443"
      - "80:80"                                       # ACME HTTP-01 challenge + HTTP→HTTPS redirect
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy-data:/data
    networks:
      - internal
    depends_on:
      - queryapigate

  # Optional - see "Shared response cache" below. Skip both this service and QUERYAPIGATE_REDIS_URL if
  # cache_ttl either isn't used or the built-in per-process cache is enough.
  redis:
    image: redis:7-alpine
    restart: unless-stopped
    networks:
      - internal

networks:
  internal:

volumes:
  queryapigate-data:
  caddy-data:
```

```caddyfile
# Caddyfile
api.example.com {
    reverse_proxy queryapigate:5000
}
```

That's it for TLS - Caddy requests and renews a real certificate automatically for `api.example.com` the first time it starts, as long as DNS already points at this host and ports 80/443 are reachable from the internet (for the ACME challenge). Swap in nginx or Traefik if you already run one of those; the only requirement is that whatever terminates TLS forwards to `queryapigate:5000` and QueryAPIGate is told how many proxy hops sit in front of it via `QUERYAPIGATE_TRUST_PROXY`, so rate limiting and redirects use the real client address and scheme rather than the proxy's.

3. Secrets (`.env`)

```shell
# .env - not committed; chmod 600
QUERYAPIGATE_API_KEY=<a long random string>
PROD_DB_PASSWORD=<your database password>
```

`QUERYAPIGATE_API_KEY` is the admin key - full access, sent as the `X-API-Key` header. Generate a real one (`openssl rand -hex 32`), not a word you'll remember; create scoped keys for anything that only needs to run queries (`POST /api/v1/api-keys` , from the admin UI or the API - see API.md). Then give each person who manages the server their own admin token and role, and keep the shared key as a break-glass key - `docker compose exec queryapigate queryapigate admins create alice --role owner` creates the first owner (API.md).

A connection's own password shouldn't be stored as plain text - give it as a reference to an environment variable instead (in the Console's connection form, or the API), resolved from the container's environment (i.e. anything in `.env` above):

```json
{"name": "prod", "db": "postgres", "host": "db.internal", "user": "app",
 "password": "${PROD_DB_PASSWORD}", "database": "app", "active": true}
```

`QUERYAPIGATE_SECRET_KEY` (a Fernet key) additionally encrypts every literal connection password *at rest* in `queryapigate.db` itself, independent of the `${VAR}` convention above - worth setting if the volume or its backups might be read by someone who shouldn't see connection credentials. Generate one and add it to `.env` (keep a copy in your secrets manager - backups of the store are useless for these passwords without it):

```shell
# a Fernet key is 32 random bytes, base64url-encoded
python3 -c "import base64, os; print(base64.urlsafe_b64encode(os.urandom(32)).decode())"
```

See Keep connection passwords out of the store.

Read the full hardening checklist before going live - `QUERYAPIGATE_ALLOW_WRITES` , `QUERYAPIGATE_RATE_LIMIT` , `QUERYAPIGATE_CORS_ORIGINS` and the rest all matter for a reachable production instance and aren't repeated here.

4. First run

```shell
docker compose run --rm queryapigate queryapigate init   # writes a starter queryapigate.db into the volume
docker compose up -d
curl https://api.example.com/health
```

`init` is optional - it only adds templates - and works either before the first `up` (as above) or afterwards with `docker compose exec queryapigate queryapigate init`; the running server sees the templates at once. It does nothing on a store that already has connections.

`init` writes one template connection per supported database type, all inactive - activate the ones you need from the admin UI (`https://api.example.com/console`) or `PATCH /api/v1/connections/{name}`. Do **not** run `queryapigate examples load` against a production instance; it's meant for trying the product, seeds a throwaway SQLite database and five example API keys, and is documented as such in EXAMPLES.md.

5. Persistent data, backups and restores

Everything QueryAPIGate owns - connections, saved queries and their versions, run history, API keys, roles and the audit log - lives in its store: `queryapigate.db` on the `/data` volume, or a PostgreSQL database if you set one up (section 8). Back up the store, and separately the few secrets it depends on (below). Then restore one now and then: a backup nobody has restored is not a backup.

The SQLite store

`queryapigate backup` copies it while the server keeps running. It uses SQLite's own backup API, so the copy is one consistent moment and includes changes still in the write-ahead log (`queryapigate.db-wal`) - which a plain `cp` of the file can miss:

```shell
docker compose exec queryapigate queryapigate backup /data/backup.db --force
docker compose cp queryapigate:/data/backup.db "./backups/queryapigate-$(date +%F).db"
docker compose exec queryapigate rm /data/backup.db
```

Run that on a schedule (host cron, or a small sidecar with its own) and ship the result off the host.

To restore, stop the server, put the backup in place as `queryapigate.db` - removing the `-wal` and `-shm` files, which belong to the store being replaced - and start it again:

```shell
docker compose stop queryapigate
docker compose run --rm -v ./backups:/backups queryapigate sh -c \
  'cp /backups/queryapigate-2026-10-04.db /data/queryapigate.db && rm -f /data/queryapigate.db-wal /data/queryapigate.db-shm'
docker compose start queryapigate
```

The PostgreSQL store

Back it up with `pg_dump`, like any other PostgreSQL data. On a managed database whose snapshots or point-in-time recovery already cover the whole database, that covers QueryAPIGate's store too. To back up just the store, dump its schema - `public`, or the one `?options=-csearch_path%3D...` names in `QUERYAPIGATE_DATABASE_URL`; `queryapigate backup` prints this command with the right schema filled in:

```shell
pg_dump --format=custom --schema=public --no-owner --no-privileges \
  --file="queryapigate-$(date +%F).dump" "$QUERYAPIGATE_DATABASE_URL"
```

`pg_dump` reads one snapshot, in a single transaction, so it is consistent and safe to run while QueryAPIGate is serving; runs recorded after it starts are simply not in it. Use a `pg_dump` of the same major version as the server, or newer.

To restore, stop every QueryAPIGate process using the store, restore into an empty database (or one where that schema has been dropped), and start them again:

```shell
pg_restore --no-owner --no-privileges --dbname="$QUERYAPIGATE_DATABASE_URL" "queryapigate-2026-10-04.dump"
```

The schema is recreated under its original name, so the URL's `search_path` (if any) must name the same schema.

What a backup of the store does not contain

- **`QUERYAPIGATE_SECRET_KEY`**, if you set it: connection passwords are stored encrypted with it. A store restored without the same key has every encrypted password unreadable - each connection must be given its password again. Keep the key in your secrets manager, with the `.env` it lives in. After a key rotation (`queryapigate rekey`), a backup taken before it still needs the old key: list it after the new one to restore it, then run `rekey` again.
- **`QUERYAPIGATE_API_KEY`**, the admin key, and any environment variable a connection refers to (`"password": "${WAREHOUSE_PASSWORD}"`). Scoped API keys *are* in the store - as hashes, so they keep working after a restore, but a lost secret can't be recovered from a backup: issue a new key instead.
- **The databases QueryAPIGate connects to** - back those up in their own right. The Redis response cache, if used, is disposable: it refills.
- **Run history and the event log, with a run store** (`QUERYAPIGATE_RUN_STORE_URL`, ADR 0029): they're in ClickHouse then, not the metadata store. Back it up with ClickHouse's own tools if history must survive; the store restored without it starts with none.

A restore from an older release's store is upgraded on first start, as any upgrade is; one from a newer release is refused rather than run on. `tests/test_backup_restore.py` does all of this on every CI run: it fills a store, backs it up, loses it, restores it, and checks the admin API sees exactly what it saw before.

6. Observability

`/metrics` (Prometheus text format) and the admin UI's own **Metrics** tab need no setup - see API.md's Observability section for the full metric list and what each one means. For real history, trends and alerting, add a Prometheus scrape target and import the bundled Grafana dashboard:

```yaml
# prometheus.yml
scrape_configs:
  - job_name: queryapigate
    authorization: {credentials_file: /etc/prometheus/queryapigate-metrics-token}   # with
QUERYAPIGATE_METRICS_TOKEN
    static_configs:
      - targets: ["queryapigate:5000"]
```

Then *Dashboards → New → Import* in Grafana, uploading `documentation/grafana-dashboard.json` from this repo - panels for request/query rate and latency (p50/p95/p99), error rate, rows returned, active queries, pool occupancy and rate-limit rejections, built on exactly the metric names `/metrics` already exposes. `/metrics` needs no API key - a scraper shouldn't hold one. It's open to whoever can reach the server, until you set `QUERYAPIGATE_METRICS_TOKEN`: then only a request with `Authorization: Bearer <the token>` - and an administrator, the Console's Metrics screen - reads it. Set it whenever the server can be reached from outside your network: the labels name your API keys, queries and connections.

With several instances, scrape each one - every process keeps its own counters, as Prometheus expects - and the bundled dashboard sums across them. The Console's Metrics screen shows only the instance that served it.

7. Shared response cache (optional)

A saved query's `cache_ttl` is served from an in-process cache by default - zero setup, but wiped on every restart. If a query is hit often enough that a warm cache actually matters, point it at the `redis` service in the compose file above instead:

```shell
# .env
QUERYAPIGATE_REDIS_URL=redis://redis:6379/0
```

See Shared response cache for what changes (a Redis outage degrades to a cache miss, never a failed request) and browse what's actually cached from the admin UI's **Caching** tab.

8. Shared metadata store (optional)

By default connections, saved queries, keys, roles, history and the audit log live in `queryapigate.db` on the `/data` volume - see section 5 for backing it up. To keep them in PostgreSQL instead (a managed database you already back up, or a store several instances share), add to `.env`:

```shell
# .env
QUERYAPIGATE_DATABASE_URL=postgresql://queryapigate:<password>@db.internal:5432/queryapigate
```

then copy the existing store across once, before switching traffic:

```shell
docker compose run --rm queryapigate queryapigate migrate-to-postgres
```

The image already includes the PostgreSQL driver. See Shared metadata store for what it changes and what it doesn't.

How many PostgreSQL connections. Each process holds one connection to the store per request thread, plus one for its history writer: about `threads + 1` per instance (9 with the image's 8 threads), plus one per `queryapigate events` and `queryapigate mcp` process. Three instances and an events server need about 30 - well inside PostgreSQL's default `max_connections = 100`, which the databases QueryAPIGate *queries* don't share. For many more instances, put PgBouncer in front in **session** pooling mode: the store's transactions take advisory locks, which transaction pooling would break.

Which instances are running. `GET /api/v1/instances` (admin key) lists every process using the store - role, host, version, whether it shares rate limits through Redis - seen in the last 90 seconds; each refreshes itself as it serves requests and health checks. Two **Alerts** watch it: several instances without Redis (`instances_not_shared`), and instances on different versions (`instances_versions_differ`), expected only during a rolling upgrade.

9. Live events for many clients (optional)

The main server's own `GET /events` holds a request thread per open stream, so it serves only a few (see Live events). For apps and phones, add the asyncio events server as a second service from the same image, and route `/events` to it:

yaml

```yaml
# docker-compose.yml - next to the queryapigate service
  events:
    image: ghcr.io/anantharajuc/queryapigate:1.2.3     # the same image and version as queryapigate
    restart: unless-stopped
    command: ["queryapigate", "events", "--host", "0.0.0.0"]
    env_file: .env                                     # the same store: QUERYAPIGATE_DATABASE_URL, or
...
    environment:
      QUERYAPIGATE_TRUST_PROXY: "1"
    volumes:
      - queryapigate-data:/data                        # ... the same /data volume, for SQLite
    healthcheck:                                       # the image's own check probes port 5000
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:5002/health', timeout=3)"]
    networks:
      - internal
```

caddyfile

```
# Caddyfile
api.example.com {
    reverse_proxy /events* events:5002
    reverse_proxy queryapigate:5000
}
```

Caddy streams `text/event-stream` responses without buffering. With nginx, use `proxy_buffering off;` and a `proxy_read_timeout` above the 15-second keepalive for that location. The events server sees runs from every instance sharing the store - one events service is enough for several `queryapigate` replicas.

Scaling out

Two shapes are supported:

	ONE INSTANCE	SEVERAL INSTANCES
Store	SQLite on a volume, or PostgreSQL	**PostgreSQL** (section 8)
Redis	optional (shared cache across restarts)	**required** - rate limits and the response cache (section 7)
Live events	`GET /events` on the instance	`queryapigate events` (section 9), routed by the proxy
Availability	restart on failure	the load balancer skips an instance that fails its health check

With a PostgreSQL store and Redis, everything an instance owns is shared - connections, saved queries and versions, keys, roles, history, the audit log, rate limits, the cache - so instances are interchangeable and a load balancer can send any request to any of them. `tests/test_multinode.py` checks this on every CI run with three real servers.

The reference deployment is `deploy/scale-out/`: three instances, PostgreSQL, Redis, the events server and Caddy load-balancing them, in one Compose file. Run end to end before it was published: requests rotated across the three instances, a key's `4/minute` limit allowed exactly four calls in total across them, live events arrived through `/events`, and with one instance stopped twelve calls in a row still succeeded - Caddy retried them on the others. Walkthrough: Run several instances.

One worker per container, more containers for more capacity. The image runs gunicorn with one worker and eight threads, and that stays right when scaling out: each process keeps its own `/metrics`, and Prometheus scraping a container with several workers would reach one of them at random. For more headroom inside one container, raise the threads - `gthread` keeps a slow request from blocking the others - with `QUERYAPIGATE_THREADS` (from 0.43; the pool follows it). Before 0.43 this took a `command:` override with `--threads`, which still works: a flag, or `GUNICORN_CMD_ARGS`, beats the setting.

```yaml
    environment:
      QUERYAPIGATE_THREADS: "16"
      QUERYAPIGATE_ACCESS_LOG: "off"   # when the proxy in front logs requests already
```

How many threads, and how much CPU to give each container: Tune a deployment for throughput.

What stays per instance: `/metrics` (scrape each instance; the bundled dashboard sums them), the Console's Metrics screen, and the main server's own `GET /events` (use `queryapigate events`). `GET /api/v1/instances` lists the instances; an `instances_not_shared` alert means some of them run without Redis.

Several instances without a PostgreSQL store - each with its own SQLite volume - are separate servers with their own keys and connections, not one service. For simple redundancy without PostgreSQL, run one active instance and a standby restored from backups.

Kubernetes (Helm)

> **Stable** since 0.30, under the compatibility promise (what that means). Values may still be added; none is renamed or removed except through the deprecation process. Limits: chart X.Y.Z runs QueryAPIGate X.Y.Z; it makes no PostgreSQL or Redis of its own; and its NetworkPolicy is off until you give it your cluster's rules.

The chart is published with every release, at the server's version: chart 0.26.0 installs QueryAPIGate 0.26.0.

```shell
helm install queryapigate oci://ghcr.io/anantharajuc/charts/queryapigate --version 0.26.0 \
  -n queryapigate --set existingSecret=queryapigate
```

It has the two shapes above. `store.type: sqlite`, the default, runs one pod with `queryapigate.db` on a PersistentVolumeClaim. The claim is kept on `helm uninstall`. Upgrades replace that pod rather than roll it: the old pod stops before the new one starts, so expect a few seconds of downtime. `store.type: postgres` runs `replicas` pods on the PostgreSQL store with Redis. They roll one at a time, and a new pod must be ready before an old one stops (`maxUnavailable: 0`). A PodDisruptionBudget keeps all but one up during node drains.

CONCERN	HOW THE CHART HANDLES IT
Secrets	`existingSecret`, a Secret you manage, becomes the pods' environment: `QUERYAPIGATE_API_KEY`, `_SECRET_KEY`, `_DATABASE_URL`, `_REDIS_URL`, `_AUDIT_KEY`, … Or `secret.create: true` with `secret.values`, which then sit in your values file.
Other settings	`config`: any `QUERYAPIGATE_*` or `OTEL_*` variable from Installation and setup.
Refusals	`helm install` fails, before anything changes, on more than one pod on SQLite, more than one without Redis, PostgreSQL without a database URL, a secret setting in `config`, or a value not in `values.schema.json`.
Routing	`ingress.*` sends `/events` to the events server (`events.enabled`) and the rest to the API pods, and sets `QUERYAPIGATE_TRUST_PROXY=1`. `mcp.enabled` runs the MCP server; `mcp.ingress.host` gives it its own host. The default annotations are ingress-nginx's no-buffering for `/events`; replace them for another controller.
Pod security	Non-root (UID 1000), a read-only root filesystem, every capability dropped, the RuntimeDefault seccomp profile, no service account token. The server writes only to `/data` and `/tmp`.
Observability	`serviceMonitor.enabled` for the Prometheus Operator (section 6), sending the token `/metrics` needs: `metricsAuth` makes one in a Secret of the chart's own, kept across upgrades - or name yours, `metricsAuth.existingSecret`, where the chart is rendered without the cluster (`helm template`, Argo CD). Tracing through `config` (`OTEL_EXPORTER_OTLP_ENDPOINT`).
Network	`networkPolicy.enabled`: in only from `networkPolicy.ingressFrom`, out only to `networkPolicy.egress` and DNS. List your databases, Redis and the SIEM there.
Scale	`autoscaling.enabled`, CPU-based, PostgreSQL shape only.
Secrets managers	`${vault:...}`, `${aws-sm:...}` and short-lived database credentials (Secrets from files and secrets managers; AWS's experimental): `vaultToken.enabled` for Vault's Kubernetes auth, `serviceAccount.annotations` for IRSA, and the `QUERYAPIGATE_VAULT_*` / `QUERYAPIGATE_AWS_*` settings through `config`.
Configuration as code	`configAsCode.enabled` with `configAsCode.file` (or `existingConfigMap`): connections, queries, roles, keys and the rest as one file in git, applied by the first new API server pod as it starts, once per file (Configuration as code). A change to it rolls the pods; a file that can't be applied stops the rollout at its first pod. `locked: true` keeps what it manages to the file.

Two things to know:

- **Changing a Secret you manage doesn't restart the pods**, since the chart can't see inside it. Run `kubectl rollout restart` on each Deployment. Changes to `config`, and to a Secret the chart makes, roll the pods on `helm upgrade`.
- **Backups** are the same as above. Run `kubectl exec deploy/queryapigate -- queryapigate backup /tmp/backup.db`, then `kubectl cp` the file out. For PostgreSQL, use your database's own backups.

Every value: the chart's README. Walkthrough, run on kind: Install on Kubernetes with Helm. Plain manifests of the PostgreSQL shape, kept in step with the chart by a test: `deploy/kubernetes/`.

A Rust data plane in front of /q/

> **Experimental** - outside the compatibility promise (what that means): decided go in 0.44 (ADR 0028) and shipped in the image and the chart (ADR 0030); what it serves, its settings and its headers may change before it graduates.

A second process, `queryapigate-dataplane` in the image, can take the public port and serve saved-query reads itself, from plans the Python server gives it, forwarding everything else to Python unchanged. For the calls it serves it measured 3-9x the calls a second per core, with every run still recorded (Performance, v5).

It runs as a second container from the same image, sharing the first one's network: Python on `127.0.0.1:5001` (`QUERYAPIGATE_BIND`), the data plane on port 5000, both with the same `QUERYAPIGATE_DATAPLANE_SECRET` (at least 16 characters) and settings. The Helm chart does that with `dataplane.enabled: true`. One gunicorn worker records about 17,000 runs a second: without a run store (guide 65), keep a pod's data plane near 2 CPUs and scale out with pods. It isn't for FIPS 140 deployments: its cryptography isn't a validated module. To turn it on, and see which calls it serves: Try the Rust data plane.

A ClickHouse run store (prototype)

> **Experimental** - outside the compatibility promise (what that means): a prototype (ADR 0029, measured and decided go). Its tables, its event ids and how it's switched on may change before it graduates.

With `QUERYAPIGATE_RUN_STORE_URL=https://user:password@host:8443/database` set on every instance, run history, the event log and per-minute and per-hour usage rollups are kept in ClickHouse (24.8 or later) instead of the metadata store - for a data plane's rates, where every run recorded is billions of rows a day. Keys, queries and the audit log stay where they are. History is kept by time (`QUERYAPIGATE_HISTORY_RETENTION_DAYS`, 30 days unless set), and live events arrive about a second later than from the metadata store. Every instance must name the same server and database: the store records which one holds its runs, and an instance that disagrees refuses to start. The users, the data plane's own insert-only one, copying history across and switching back: Keep run history and the event log in ClickHouse.

Upgrading

1. Read the Changelog entry for the target version, especially anything under a "Breaking" or "Upgrading" heading.
2. Back up first - always, even for a patch version.
3. Bump the pinned tag in `docker-compose.yml`, then `docker compose pull && docker compose up -d`.
4. Check `docker compose logs queryapigate` and `curl .../health`.

The `queryapigate.db` schema is migrated automatically on the container's first start against it - nothing to run by hand. Every release since 0.7 is tested this way: a store each one built through its own API is started under the current code, which must read back its connections, queries and their versions, run history, keys and roles (`tests/test_upgrades.py`). A store this version can't safely run on - one with a column missing, or one a *newer* release has already upgraded - stops startup with a message naming the problem and what to do, before anything is changed; it never starts and then fails on first use. Going back to an older release is not supported in place: restore the backup from step 2.

Rolling upgrades (several instances on one store)

Replace instances one at a time - no downtime - because of this rule: **within 1.x, a release changes the store only additively** (new tables, new nullable columns, new indexes; never a rename, a drop or a changed meaning). **One exception, before 1.x: the upgrade to 0.38** (workspaces, ADR 0023) changes the store's primary keys, so stop every 0.37 instance before the first 0.38 one starts - see the changelog. So while some instances run the new release and have upgraded the store, the ones still on the previous release keep working against it. `tests/test_rolling_upgrade.py` checks exactly that on every CI run: the previous release from PyPI keeps serving, writing and recording runs on a store the current code has just upgraded, and each sees the other's changes.

1. Back up the store.
2. Start one instance on the new version - it upgrades the store on its first start - and check its `/health`.
3. Replace the others one by one. `GET /api/v1/instances` shows who is on which version meanwhile.
4. Replace `queryapigate events`, if you run it, with the instances. From 0.33 it streams the event log (ADR 0020), and an instance still on 0.32 writes its runs to history only - so they aren't events until it's upgraded too.

Two limits:

- **An instance can't *start* on the previous release once the store is upgraded** - it refuses, naming the newer release. If one restarts mid-upgrade, bring it back on the new version.
- **Upgrade one minor version at a time** (0.15 → 0.16, not 0.15 → 0.17) when rolling: the guarantee is about the release just before. A release that can't keep it says so under **Breaking** in its changelog entry, and then all instances must be stopped before the first one starts on it.